Bases de Datos a Gran Escala

Sesión 2 · SQL (ii): DDL

Diego Sevilla Ruiz
Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es

SQL (ii): DDL

Introducción

- SQL se divide lógicamente en dos sublenguajes:
  - **DML** (*Data Manipulation Language*), visto en la sesión anterior
  - **DDL** (*Data Definition Language*), el sublenguaje de definición del esquema de los datos
- En esta sesión veremos también la parte de actualización, modificación y borrado de datos de DML
- Veremos cómo usar JSON en SQL
- Finalmente, una guía de diseño de esquemas orientados a la eficiencia

Data Definition Language (DDL)

- Permite describir los metadatos (tablas y relaciones entre ellas) de la base de datos
- Tablas, dominios de los atributos y restricciones de integridad
- Al conjunto de especificaciones se le conoce como **esquema** de la base de datos
- Por lo tanto, define también la sintaxis de las operaciones de inserción (`INSERT`) y actualización (`UPDATE`)

CREATE TABLE

- La construcción principal de SQL para la creación del esquema es la sentencia `CREATE TABLE`
- Para cada tabla, especifica sus columnas, dominios de columna y restricciones

```
CREATE TABLE nombre (
  att1 tipo1 [[NOT] NULL] [UNIQUE] [DEFAULT x],
  att2 tipo2,
   ...
  PRIMARY KEY( ... ),
  FOREIGN KEY attx REFERENCES Tabla(atty)
    ON DELETE opción
    ON UPDATE opción,
   ...
);
```

`CREATE TABLE`: columnas

- Cada campo lista su tipo (se ven a continuación)
- Opcionalmente se puede especificar si la columna admite valores nulos (`NULL`) o no (`NOT NULL`)
- También se puede indicar que la columna contiene valores que no se repetirán, con `UNIQUE`
- Se le puede asignar un valor por defecto al insertar datos, con `DEFAULT x`
- No se muestra en el ejemplo, pero a un atributo también se le puede añadir `PRIMARY KEY`
- La clave primaria puede ser un dato del dominio o un número generado por la base de datos (a continuación)

Clave natural y clave subrogada

¿Qúe elemento se utiliza para identificar a cada entidad?

- Clave **natural**: un dato con significado en el dominio (correo, DNI, ISBN)
  - Puede cambiar, faltar o ser larga, y cada clave ajena la copia
- Clave **subrogada**: un número sin significado, generado al insertar, que nunca cambia
  - En MySQL, con `AUTO_INCREMENT`; así podrían ser los `Id` de Stack Overflow

```sql
CREATE TABLE Users (
  Id          INT AUTO_INCREMENT PRIMARY KEY,   -- subrogada
  Email       VARCHAR(255) NOT NULL UNIQUE,     -- natural, sigue siendo única
  DisplayName VARCHAR(40) NOT NULL
);

INSERT INTO Users (Email, DisplayName) VALUES ('ana@example.com', 'Ana');
-- No se indica Id: MySQL le asigna 1, a la siguiente fila 2 ...
```

`CREATE TABLE`: claves ajenas

- Opcionalmente, se pueden añadir restricciones de integridad con `FOREIGN KEY`
  - Lista un conjunto de atributos de la tabla (***clave ajena***)
  - Que hace referencia a otro conjunto de atributos de otra tabla (***clave***)
  - Opcionalmente se puede especificar qué sucede cuando el elemento referenciado se elimina (`ON DELETE`) o se le cambia el valor (`ON UPDATE`)
  - Se estudian las opciones con detalle a continuación

Creación de tablas a partir de consultas

- La construcción `CREATE TABLE` también permite crear tablas a partir de consultas
- La sintaxis es `CREATE TABLE t AS SELECT ...`:

```
CREATE TABLE T AS SELECT Id,Title FROM Posts;
```

- Copia columnas y filas, pero no la clave primaria, los índices ni las claves ajenas
- La alternativa habitual en los cuadernos es crear la tabla con su DDL completo y rellenarla con `INSERT ... AS SELECT`

Juego de caracteres e intercalación

- El **juego de caracteres** decide qué caracteres se pueden almacenar; la **intercalación** (*collation*), cómo se comparan y ordenan
- Se fijan para la base, la tabla o la columna:

```
CREATE DATABASE stackoverflow
  CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_as_ci;
```

- `utf8mb4` admite todo Unicode (el antiguo `utf8` de MySQL no)
- `_as_ci` : distingue acentos (*accent sensitive*) e ignora mayúsculas (*case insensitive*): `'ánimo' = 'animo'` es falso y `'Ánimo' = 'ánimo'` es cierto
- La intercalación afecta a `WHERE` , `ORDER BY` , `GROUP BY` , `UNIQUE` y a los índices: es una decisión de modelado, no un detalle de instalación

Tipos de datos: caracteres y enteros

- Caracteres:
  - `char(n)` un conjunto de `n` caracteres exactamente
  - `varchar(n)` un conjunto variable de como máximo `n` caracteres
  - `text` un conjunto de caracteres sin límite; no es parte del estándar SQL, pero lo soportan casi todos los SGBDR
- Números enteros: `integer`, `smallint`, `bigint`

Tipo	Tamaño	Rango
`smallint`	2 bytes	−32 768 a +32 767
`integer`	4 bytes	−2 147 483 648 a +2 147 483 647
`bigint`	8 bytes	−9 223 372 036 854 775 808 a +9 223 372 036 854 775 807

Tipos de datos: números, mapas de bits y fechas

- Números de coma flotante (aproximados): `FLOAT` (4 bytes) y `DOUBLE` / `DOUBLE PRECISION` (8 bytes)
  - En MySQL, `REAL` es sinónimo de `DOUBLE` (salvo con el modo `REAL_AS_FLOAT`)
  - La forma `FLOAT(m,d)` / `DOUBLE(m,d)` es una extensión de MySQL obsoleta desde 8.0.17: no conviene usarla
- Números exactos: `DECIMAL(m,d)` / `NUMERIC(m,d)`, donde `m` es el número total de dígitos y `d` el de decimales; es el tipo adecuado para importes
- Mapas de bits: `BIT(n)`
- Fechas: `DATE`, `TIME`, `DATETIME`, `TIMESTAMP`, `YEAR` — se ven en detalle a continuación; `INTERVAL` no es un tipo de columna en MySQL, sino parte de la aritmética de fechas

Fechas

- La gestión de fechas cambia de un SGBDR a otro; la vemos para MySQL
- Para ver los valores, podemos fijarnos en su «valor cero»:

Tipo de dato	Valor cero
`DATE`	`'0000-00-00'`

`TIMESTAMP` contiene una fecha compacta en el rango `'1970-01-01 00:00:01'` UTC a `'2038-01-19 03:14:07'` UTC

Las fechas cero sólo se aceptan si el `sql_mode` lo permite: en MySQL 8 el modo por defecto (estricto, con `NO_ZERO_DATE` y `NO_ZERO_IN_DATE`) las rechaza

Fechas: funciones

- MySQL ofrece funciones `YEAR()`, `MONTH()`, etc. para extraer datos particulares
- Funciones de búsqueda con intervalos (similares a ANSI-SQL): `INTERVAL`, `BETWEEN`, etc.
- `CURRENT_DATE()`, `CURRENT_TIME()`, `CURRENT_TIMESTAMP()`, etc. devuelven el tiempo actual con el tipo correspondiente

Posts desde hace una semana:

```
SELECT * FROM Posts WHERE CreationDate ≥ CURRENT_DATE() - INTERVAL 1 WEEK;
```

(se puede usar `WEEK`, `DAY`, `HOUR`, etc.)

Fechas: rangos

```
SELECT * FROM Posts WHERE CreationDate
    BETWEEN CURRENT_DATE() - INTERVAL 1 MONTH
        AND CURRENT_DATE() - INTERVAL 1 WEEK;
```

Entre fechas específicas: las cuatro semanas anteriores, sin contar la última

El operador `CAST()`

- SQL permite cambiar el tipo de una expresión con `CAST()`
- Los tipos permitidos son los vistos, aunque hay cierta variación
- Por ejemplo, para un `INTEGER` siempre hay que decir si es `SIGNED` o `UNSIGNED`
- También se permite el tipo `CHAR`:

```
> SELECT CONCAT('ABC',CAST(1.1 AS CHAR(200))) as ABC;
ABC1.1
> SELECT CAST('1.1' as UNSIGNED) AS C;    -- entero: trunca y avisa
1
> SELECT CAST('1.1' as DECIMAL(10,5)) AS C;
1.10000
```

Aseguramiento de la integridad

- Las partículas `FOREIGN KEY` de `CREATE TABLE` sirven para especificar las restricciones de integridad
  - **Integridad referencial**: no se puede insertar un valor en una clave ajena que no exista en la tabla referenciada
- En cada tipo de modificación (`UPDATE` y `DELETE`) hay una posibilidad de trasladar cambios en cascada
- Estas modificaciones se refieren a la eliminación o actualización de **una fila que referencia a una fila eliminada o actualizada**

```
FOREIGN KEY attx REFERENCES Tabla(atty)
    ON DELETE <acción>
    ON UPDATE <acción>
```

Aseguramiento de la integridad: opciones

- `NO ACTION` (por defecto) o `RESTRICT`. No permite la modificación: el gestor lanza un error y no deja hacer el cambio, para mantener la consistencia/integridad. En InnoDB ambas son equivalentes (se comprueban de inmediato)
- `CASCADE`. Elimina (o actualiza) todas las filas que referencian a la fila modificada (las que tienen su clave ajena apuntando a esta fila)
- `SET NULL`. Pone a nulo la clave ajena de las filas que la referencian (la columna debe admitir `NULL`)
- `SET DEFAULT`. Definida en el estándar: pone el valor por defecto. MySQL la reconoce, pero InnoDB rechaza la tabla que la usa

Ejemplo de cascada

```
> CREATE TABLE Factura
  (Id INT PRIMARY KEY, Cliente TEXT, Total DECIMAL(10,2));
> CREATE TABLE ItemFactura
  (Id INT  , FactId INT, Nombre TEXT,
   Cantidad INT, Precio DECIMAL(10,2),
     PRIMARY KEY(Id,FactId),
     FOREIGN KEY (FactId) REFERENCES Factura(Id)
       ON DELETE CASCADE);

> INSERT INTO Factura VALUES (5, 'Pedro', 200.0);
> INSERT INTO ItemFactura VALUES (1, 5, 'Tuerca', 2, 100);
```

Ejemplo de cascada: efecto del `DELETE`

```
> SELECT * FROM Factura;
Id    Cliente         Total
────  ──────────────  ──────
5     Pedro           200.00
> SELECT * FROM ItemFactura;
Id    FactId  Nombre  Cantidad  Precio
────  ──────  ──────  ────────  ──────
1     5       Tuerca  2         100.00

> DELETE FROM Factura WHERE Id = 5;
> SELECT COUNT(*) FROM ItemFactura;
0
```

Al borrar la factura, `ON DELETE CASCADE` arrastra sus líneas

Alteración del esquema

- Una vez creada la tabla se puede alterar: añadir columnas, restricciones, etc.
- Esto tiene un impacto en la base de datos (consultas en ejecución, bloqueos, recálculos como en el caso de añadir o modificar índices, etc.)
- Algunos cambios obligan a reconstruir la tabla entera: en una tabla grande es una operación costosa, igual que crear un índice o reparticionar

Añadir columnas:

```
ALTER TABLE tabla ADD columna tipo [DEFAULT valor];
```

Alteración del esquema: columnas

Eliminar columnas:

```
ALTER TABLE tabla DROP columna;
```

Modificar una columna:

```
ALTER TABLE tabla MODIFY columna tipo;
```

Cambiar el nombre y definición de una columna:

```
ALTER TABLE tabla CHANGE columna_anterior columna_nueva tipo;
```

Alteración del esquema: restricciones

Añadir una clave primaria:

```
ALTER TABLE tabla ADD PRIMARY KEY (columna);
```

Añadir una clave ajena:

```
ALTER TABLE tabla ADD FOREIGN KEY (columna) REFERENCES tabla_(columna_);
```

Eliminar una tabla:

```
DROP TABLE tabla;
```

Alteración del esquema: columnas calculadas

También se pueden añadir campos calculados (también en `CREATE TABLE`):

```
ALTER TABLE tabla ADD columna <tipo>
  GENERATED ALWAYS AS (<expresión>) VIRTUAL | STORED;
```

- `VIRTUAL` (por defecto) indica que el valor se calcula al leer la fila y no ocupa espacio en la tabla
- `STORED` indica que se calcula al insertar o actualizar la fila, por lo que ocupa espacio en disco
- En InnoDB ambas admiten índices secundarios (el índice sobre una `VIRTUAL` sí guarda el valor); sólo una `STORED` puede formar parte de la clave primaria
- La expresión debe ser determinista: no puede usar `NOW()`, `RAND()` ni consultas

Columnas calculadas: ejemplo

Generar los *tags* separados por comas:

```
ALTER TABLE Posts ADD COLUMN tags_csv TEXT GENERATED ALWAYS AS
  (REPLACE(REPLACE(REPLACE(Tags, '><', ','),'<',''),'>','')) VIRTUAL;
```

O, como se verá más tarde, como un *array* JSON:

```
ALTER TABLE Posts ADD COLUMN tags_array JSON GENERATED ALWAYS AS
  (CAST(CONCAT('[',
               REPLACE(REPLACE(REPLACE(Tags, '><', '","'),'<','"'),'>','"'),
               ']')
        AS JSON)
  ) STORED;
```

Ejemplo Stack Overflow

```
CREATE TABLE Posts (
  Id INT,
  PostTypeId INT,
  AcceptedAnswerId INT,
  ParentId INT,
   ...
  OwnerUserId INT,
   ...
  PRIMARY KEY(Id),
  FOREIGN KEY (ParentId) REFERENCES Posts(Id),
  FOREIGN KEY (OwnerUserId) REFERENCES Users(Id)
);
```

Ejemplo Stack Overflow (ii)

```
CREATE TABLE Comments (
  Id INT,
  PostId INT,
   ...
  UserId INT,
  PRIMARY KEY(Id),
  FOREIGN KEY (PostId) REFERENCES Posts(Id),
  FOREIGN KEY (UserId) REFERENCES Users(Id)
);
```

Vistas

- Las vistas son consultas parciales que se pueden reutilizar después
- Representan patrones que se pueden sustituir en la consulta donde se usan
- Es decir, donde se usa la vista se podría sustituir por su definición
- Se crean con `CREATE OR REPLACE VIEW v AS SELECT ...`
- Dependiendo del SGBDR se generan vistas persistentes (guardadas en disco) o en memoria

Vistas: ejemplo

Sólo preguntas (o sólo respuestas):

```sql
CREATE OR REPLACE VIEW Questions AS SELECT * FROM Posts WHERE PostTypeId = 1;
```

Después se puede utilizar como si fuera una tabla:

```sql
SELECT * FROM Questions WHERE Title LIKE '%Java%';
```

Las vistas se pueden utilizar para eliminar y para insertar valores, siempre que no estén formadas por más de una tabla con `JOIN`

Expresiones de tabla comunes (CTE)

Una CTE (*Common Table Expression*) da nombre al resultado de una consulta durante una sola sentencia:

```sql
WITH PreguntasPorAutor AS (
  SELECT OwnerUserId, COUNT(*) AS NumPreguntas
  FROM Posts
  WHERE PostTypeId = 1
  GROUP BY OwnerUserId
)
SELECT * FROM PreguntasPorAutor WHERE NumPreguntas ≥ 10;
```

- Permite dividir una consulta compleja en pasos legibles
- Se pueden definir varias CTE separadas por comas y hacer que unas usen otras
- No crea una tabla ni una vista persistente
- El nombre no implica que el motor materialice el resultado: lo decide el optimizador

Varias CTE en una consulta

```sql
WITH Preguntas AS (
  SELECT Id, OwnerUserId FROM Posts WHERE PostTypeId = 1
), Respuestas AS (
  SELECT ParentId, COUNT(*) AS NumRespuestas
  FROM Posts WHERE PostTypeId = 2
  GROUP BY ParentId
)
SELECT p.Id, p.OwnerUserId, COALESCE(r.NumRespuestas, 0)
FROM Preguntas AS p
LEFT JOIN Respuestas AS r ON r.ParentId = p.Id;
```

Cada CTE debe tener una granularidad clara. Aquí `Respuestas` produce como máximo una fila por pregunta antes de realizar la unión

CTE recursivas: `WITH RECURSIVE`

Una CTE recursiva une dos partes con `UNION ALL`:

```
WITH RECURSIVE Nombre AS (
  SELECT  ...                          -- miembro ancla: una sola vez
  UNION ALL
  SELECT  ...
  FROM Origen JOIN Nombre ON  ...   -- miembro recursivo
)
SELECT * FROM Nombre;
```

- El **ancla** aporta las filas de partida
- El **miembro recursivo** vuelve a nombrar la propia CTE y se reevalúa hasta que una vuelta no añade filas nuevas
- Sirve para recorrer jerarquías (árboles, hilos, categorías) y necesita una condición que acote la recursión: el límite `cte_max_recursion_depth` (1000) es una red de seguridad, no la parada

CTE recursiva: el hilo de una pregunta

```
WITH RECURSIVE PostThread AS (
  SELECT Id, ParentId, Id AS RootId, 0 AS Depth
  FROM Posts
  WHERE Id = 12345              -- la pregunta de partida

  UNION ALL

  SELECT p.Id, p.ParentId, t.RootId, t.Depth + 1
  FROM Posts p
  JOIN PostThread t ON p.ParentId = t.Id
  WHERE t.Depth < 2             -- guarda de terminación
)
SELECT * FROM PostThread ORDER BY Depth, Id;
```

`Depth 0` es la pregunta de partida; cada vuelta añade las respuestas cuyo `ParentId` apunta a una fila ya encontrada. En este *dataset* sólo hay dos niveles. El ejemplo completo está en el cuaderno de la sesión

Data Manipulation Language

Modificación de estado

INSERT

- Permite insertar filas en las tablas
- Puede opcionalmente especificar las columnas sobre las que actúa
- También, opcionalmente, pueden obtenerse los valores de una consulta

```
INSERT INTO tabla[(col1,col2, ...)] VALUES (val1, val2, ...), ... ;
```

```
INSERT INTO tabla[(col1,col2, ...)] SELECT ... ;
```

UPDATE

- Actualiza filas de una tabla
- La cláusula `SET` especifica qué columnas se modifican y los nuevos valores
- La cláusula `WHERE` selecciona las filas

```
UPDATE tabla SET att1 = valor1, att2 = valor2,  ...
WHERE  ... ;
```

`UPDATE`: valores antiguos y `JOIN`

Se puede utilizar el valor antiguo de los campos en la cláusula `SET`:

```
UPDATE tabla SET att1 = att1 + 1, att2 =  ...
WHERE  ... ;
```

También se puede utilizar otra tabla como fuente, con un `INNER JOIN` (sintaxis de MySQL; PostgreSQL, por ejemplo, usa `UPDATE ... FROM`):

```
UPDATE tabla
JOIN otra ON tabla.val = otra.otroval
SET tabla.att1 = otra.otroatt1,  ...
WHERE  ... ;
```

DELETE

- Elimina filas de una tabla
- Similar a `SELECT`, con una sola tabla en `FROM`
- La cláusula `WHERE` selecciona las filas

```
DELETE FROM tabla WHERE condición;
```

Gestión de JSON en SQL

Introducción

- Los SGBDR actuales permiten la integración con JSON, lo cual compararemos con las bases de datos documentales que veremos después
- Las tablas pueden tener columnas de tipo JSON
- Los documentos JSON pueden tener estructuras complejas
- Se pueden hacer consultas sobre los documentos JSON y sus campos
- Se pueden crear índices para acelerar las búsquedas usando campos de tipo JSON
- El estándar SQL incluye funciones SQL/JSON desde SQL:2016 y un tipo `JSON` desde SQL:2023, pero cada SGBDR tiene su propia sintaxis
- El soporte de MySQL es propio del producto: el tipo `JSON` y los operadores `->` / `->>` no son la interfaz del estándar ni se trasladan tal cual a otro gestor

Esquema mínimo (MySQL)

Columna tipo JSON y columnas generadas:

```
CREATE TABLE Posts (
  Id BIGINT PRIMARY KEY,
  CreationDate TIMESTAMP,
  Score INT,
  doc JSON -- documento completo
);
CREATE TABLE Users (
  Id BIGINT PRIMARY KEY,
  doc JSON
);
-- Columna generada para indexar OwnerUserId
ALTER TABLE Posts ADD COLUMN OwnerUserId BIGINT
  GENERATED ALWAYS AS (JSON_EXTRACT(doc, '$.OwnerUserId')) STORED;
CREATE INDEX idx_posts_owner ON Posts(OwnerUserId);
```

Inserción JSON

MySQL:

```
INSERT INTO Posts (Id, CreationDate, Score, doc) VALUES (
  1, NOW(), 10,
  JSON_OBJECT('Title','How to X?','Body','...','OwnerUserId',42,'Tags',JSON_ARRAY('sql','json'))
);
```

Y así queda la fila en la tabla (`SELECT * FROM Posts`):

Id	CreationDate	Score	doc	OwnerUserId
1	2026-10-07 10:15:00	10	`{"Body": "...", "Tags": ["sql", "json"], "Title": "How to X?", "OwnerUserId": 42}`	42

MySQL guarda el JSON en formato binario y reordena las claves (primero las más cortas); `OwnerUserId` se rellena sola por ser columna generada

Operadores `->` y `->>` (MySQL)

- `doc -> 'key'` devuelve JSON (tipo `JSON`)
- `doc ->> 'key'` devuelve texto (tipo `TEXT`)

```
-- JSON_EXTRACT devuelve JSON; JSON_UNQUOTE(JSON_EXTRACT) devuelve texto
SELECT JSON_EXTRACT(doc, '$.Title') FROM Posts WHERE id = 1;
SELECT JSON_UNQUOTE(JSON_EXTRACT(doc, '$.Title')) AS Title FROM Posts WHERE id = 1;
-- En MySQL se pueden usar los operadores (->, ->>)
SELECT doc->'$.Title', doc->>'$.Title' FROM Posts WHERE id = 1;
>>> "How to X?",How to X?
```

Más ejemplos

```sql
SELECT JSON_EXTRACT(doc, '$.Title') AS title_text FROM Posts WHERE id = 1;

-- Propiedad anidada (si doc contiene {"owner": {"id": 42}})
SELECT JSON_EXTRACT(doc, '$.owner.id') FROM Posts WHERE id = 1;

-- O incluso
SELECT JSON_EXTRACT(JSON_EXTRACT(doc, '$.owner'), '$.id') FROM Posts WHERE id = 1;
```

`JSON_TABLE` expande *arrays*

```sql
-- Expandir tags con JSON_TABLE
SELECT p.id, jt.tag
FROM Posts p,
     JSON_TABLE(p.doc, '$.Tags[*]' COLUMNS (tag TEXT PATH '$')) AS jt
WHERE p.id = 1;
```

`jt` por sí misma: una fila por elemento de `$.Tags`

tag
sql
json

Resultado: la fila de `Posts` se repite con cada fila de `jt`

id	tag
1	sql
1	json

Búsquedas: operador de contención (MySQL)

```sql
-- Buscar OwnerUserId
SELECT * FROM Posts WHERE JSON_UNQUOTE(JSON_EXTRACT(doc, '$.OwnerUserId')) = '42';

-- Buscar tag dentro de array (JSON_CONTAINS)
SELECT * FROM Posts WHERE JSON_CONTAINS(doc, '"sql"', '$.Tags');
```

`JSON_TABLE` (MySQL)

MySQL tiene `JSON_TABLE`, que mapea JSON a columnas:

```sql
SELECT *
FROM Posts,
     JSON_TABLE(doc, '$' COLUMNS (
       Title VARCHAR(200) PATH '$.Title',
       Body TEXT PATH '$.Body',
       OwnerUserId BIGINT PATH '$.OwnerUserId'
     )) AS jt;
```

`jt` por sí misma, para el `doc` de la fila con `Id` 1: una fila (la ruta `'$'` es el documento entero) con una columna por cada `PATH`

Title	Body	OwnerUserId
How to X?	…	42

`JSON_TABLE` (MySQL): resultado

Cada fila de `Posts` se une con las filas que genera `JSON_TABLE` a partir de su `doc`; `SELECT *` devuelve las columnas de `Posts` seguidas de las de `jt`

Id	CreationDate	Score	doc	OwnerUserId	Title	Body	OwnerUserId
1	2026-10-07 10:15:00	10	`{"Body": "…", …}`	42	How to X?	…	42

La primera `OwnerUserId` es la columna generada de `Posts`; la última, la que extrae `jt` del JSON

Posts con usuario: de dónde se parte

Dos tablas relacionales (sólo las columnas que usa el ejemplo)

`Posts`

Id	OwnerUserId	Title	Body	Score
1	42	How to X?	…	10
2	NULL	What is Y?	…	3

`Users`

Id	DisplayName	Reputation
7	Luis	320
42	Ana	1500

El post 2 no tiene propietario (`OwnerUserId` nulo, cuenta borrada)

Ejemplo: Posts con usuario (MySQL)

```
INSERT INTO Posts (Id, CreationDate, PostTypeId, OwnerUserId, doc)
SELECT
    p.Id, p.CreationDate, p.PostTypeId, p.OwnerUserId,
    JSON_OBJECT(
        'post', JSON_OBJECT(
            'Title', p.Title, 'Body', p.Body, 'Score', p.Score, ...
        ),
        'user', CASE
            WHEN u.Id IS NOT NULL THEN JSON_OBJECT(
                'Id', u.Id, 'DisplayName', u.DisplayName,
                'Reputation', u.Reputation, ...
                )
            ELSE JSON_OBJECT('Id', NULL, 'DisplayName', 'Usuario eliminado')
        END
    ) as doc
FROM Posts p
LEFT JOIN Users u ON p.OwnerUserId = u.Id;
```

Posts con usuario: el `LEFT JOIN`

`LEFT JOIN Users u ON p.OwnerUserId = u.Id` empareja cada post con su usuario; si no hay, conserva el post con `NULL` en las columnas de `u`

p.Id	p.OwnerUserId	p.Title	u.Id	u.DisplayName	u.Reputation
1	42	How to X?	42	Ana	1500
2	NULL	What is Y?	NULL	NULL	NULL

El `CASE` mira `u.Id IS NOT NULL` para decidir qué objeto `user` construir

Posts con usuario: el documento resultante

`doc` de cada fila (con `JSON_PRETTY`; MySQL ordena las claves, primero las más cortas)

Post 1

```
{
  "post": {
    "Body": "...",
    "Score": 10,
    "Title": "How to X?"
  },
  "user": {
    "Id": 42,
    "Reputation": 1500,
    "DisplayName": "Ana"
  }
}
```

Post 2

```
{
  "post": {
    "Body": "...",
    "Score": 3,
    "Title": "What is Y?"
  },
  "user": {
    "Id": null,
    "DisplayName": "Usuario eliminado"
  }
}
```

Consejos de modelado

- Si un campo va a usarse frecuentemente para filtrado u orden: mantenerlo como columna separada además del JSON
- JSON es ideal para documentos semiestructurados y variantes entre registros
- Normaliza cuando hay relaciones 1:N que se consultarán frecuentemente

Cuestiones de modelado y eficiencia

Modelado conceptual

Modelado conceptual

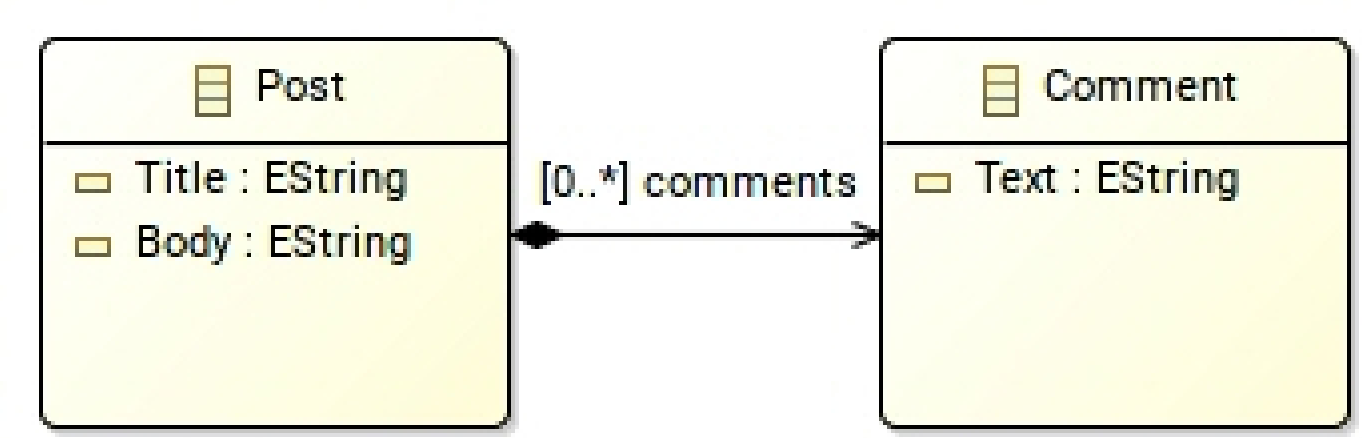
- Al modelar transformamos las entidades del «mundo» en entidades del dominio de modelado
- Existen diferentes formas de modelar; las más generalizadas:
  - Modelado basado en objetos
  - Modelado basado en entidades y relaciones (ER)
  - Modelado relacional
- Cada forma de modelado posee:
  - Unos **conceptos** (estructuras)
  - Unas **restricciones**
  - Unas **operaciones** para el acceso a los datos

Modelado conceptual: conceptos base

En general, la mayoría de las técnicas de modelado de datos se basan en:

- **Entidades**: elementos del dominio a modelar con entidad e identidad propia
- **Relaciones**: relaciones entre elementos del dominio

Modelado basado en objetos



En este modelo, el dominio se representa con **objetos** y **relaciones** de herencia, clientela y composición (contenido)

Modelado basado en objetos: clases

Correspondería a un código de definición de clases parecido a:

```
class Post:
    def __init__(self):
        self.Title = ""
        self.Body = ""
        self.Comments = []

class Comment:
    def __init__(self):
        self.Text = ""
```

Uso del modelo en un lenguaje OO

```
# Base de datos de Posts
stackoverflow = []

# A través de la página web, nuevo Post ...
# (request representa la petición web recibida)
new_post = Post()
new_post.Title = request.Title
new_post.Body = request.Body
stackoverflow.append(new_post)

# A través de la página web, nuevo comentario
# sobre un Post que se mostró en la web ...
new_comment = Comment()
new_comment.Text = request.Text
request.Post.Comments.append(new_comment)
```

Navegación

```
# Dado un Post, mostrar el texto
# de todos los comentarios
for comment in post.Comments:
    print(comment.Text)

# Número medio de comentarios por Post
nc = 0
for post in stackoverflow:
    nc = nc + len(post.Comments)
print(nc / len(stackoverflow))

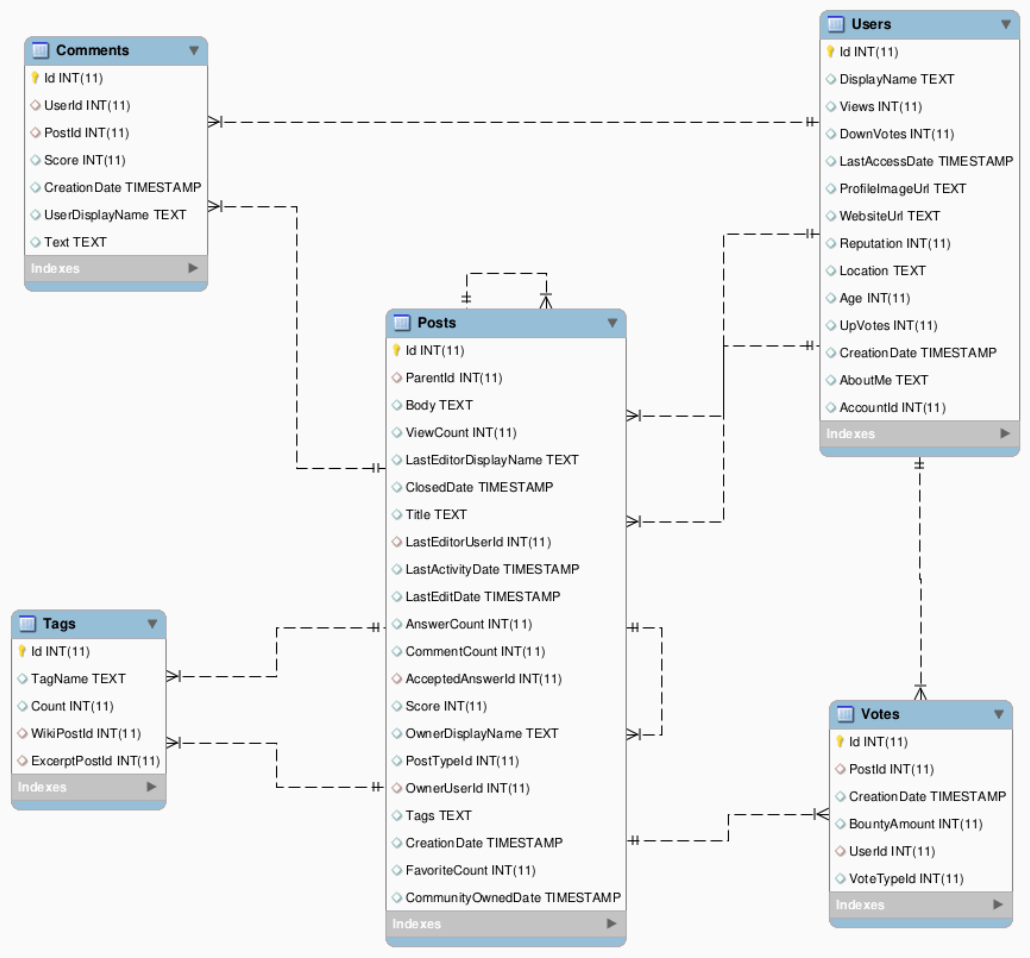
# Número medio de comentarios por Post (alt)
print(sum(len(p.Comments) for p in stackoverflow)
      / len(stackoverflow))
```

Modelado basado en objetos: valoración

- El modelado es más natural (se modelan objetos que pueden corresponder con el mundo real)
- Se incluyen relaciones de herencia, parte-de, etc.
- Se permiten relaciones de contenido, jerárquicas
- Se permiten colecciones de objetos
- El tiempo de vida (*lifespan*) de una entidad se liga a las relaciones de contenido:
  - Entidades dependientes de un «padre»: contenido
  - Entidades con vida independiente: referencia (clientela)
- Es el par **agregar/referenciar** de la sesión 1, que volverá con el modelo documental

Modelado relacional

Modelado relacional: Stack Overflow

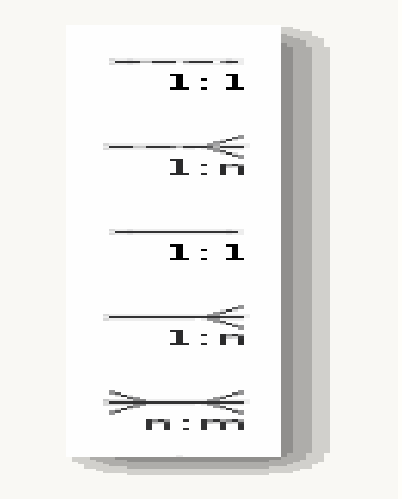


Modelado relacional

- Caso particular de modelado conceptual que utiliza las abstracciones del **modelo relacional**:
  - Tablas para describir entidades del dominio
  - Relaciones entre entidades (tablas)
  - Restricciones (claves ajenas, tipos básicos)
- El modelado también se hace en base a **entidades** y **relaciones**:
  - Siguiendo las restricciones vistas en la sesión anterior
  - Aplicando los conceptos del modelado de datos conceptual

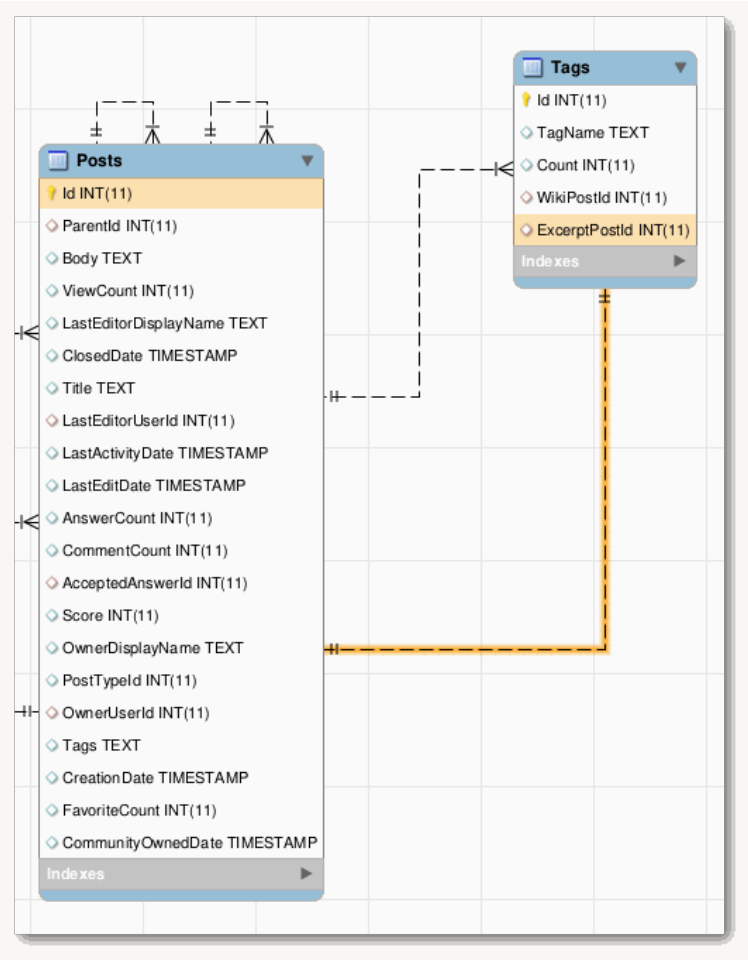
Relaciones entre entidades

- **Cardinalidad** y **dependencia**
- Cardinalidad: las relaciones pueden ser **1:1**, **1:n** o **n:m**, e indican el grado de relación de una entidad con el resto
- Dependencia (tiempo de vida):
  - ***Identifying*** (dependientes): el tiempo de vida de la entidad enlazada depende de la entidad padre
  - ***Non-identifying*** (no dependientes): las entidades tienen vida propia



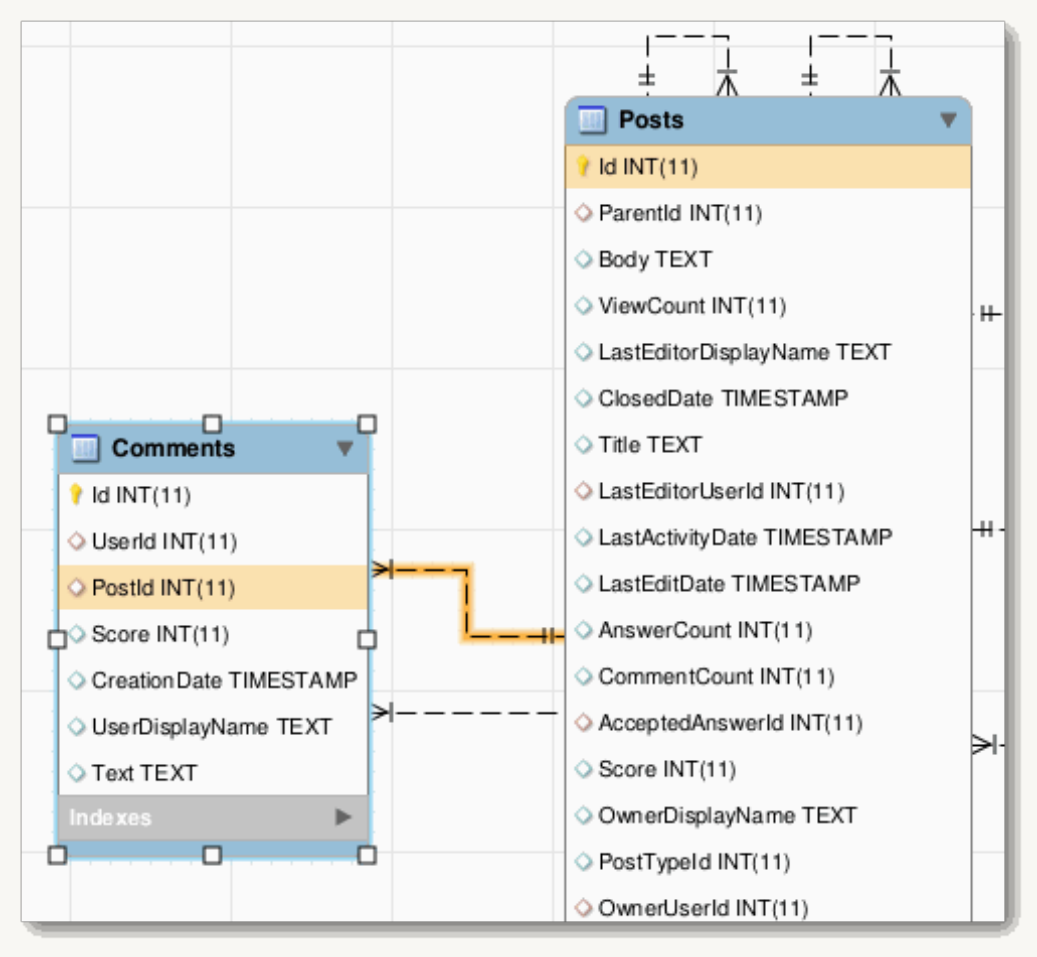
Relaciones 1:1

- **Conceptualmente** existe un Post que hace de *excerpt* (resumen, descripción) de un Tag
- Posts y Tags, con respecto a «Excerpt», es **1:1**
- En el modelado relacional equivale a una clave ajena que puede estar en una de las dos tablas o en ambas



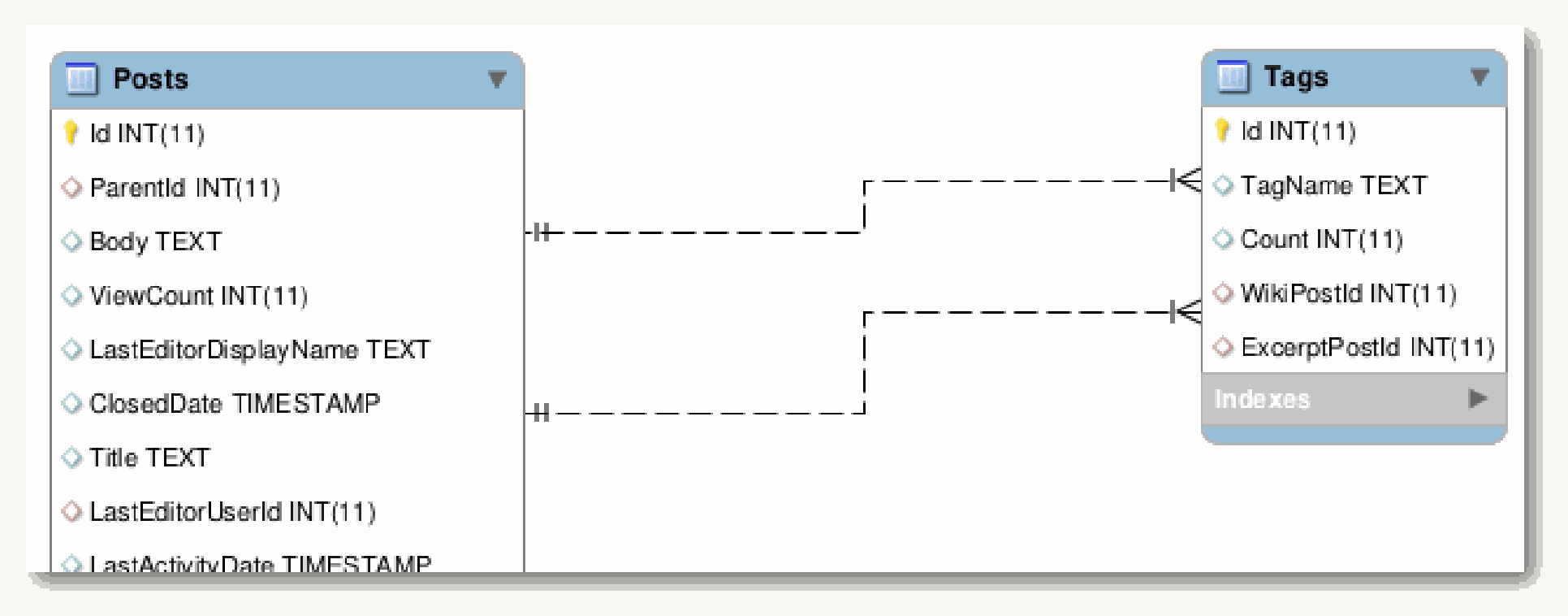
Relaciones 1:n (uno a muchos)

- **Conceptualmente** se pueden hacer **comentarios** sobre un **Post**
- Posts y Comments tienen una relación **1:n**
- Puede ser dependiente o no (se ve después)
- En el modelado relacional equivale a una clave ajena en la clase dependiente, o bien a una tabla separada (más adelante)



Relaciones n:m (muchos a muchos)

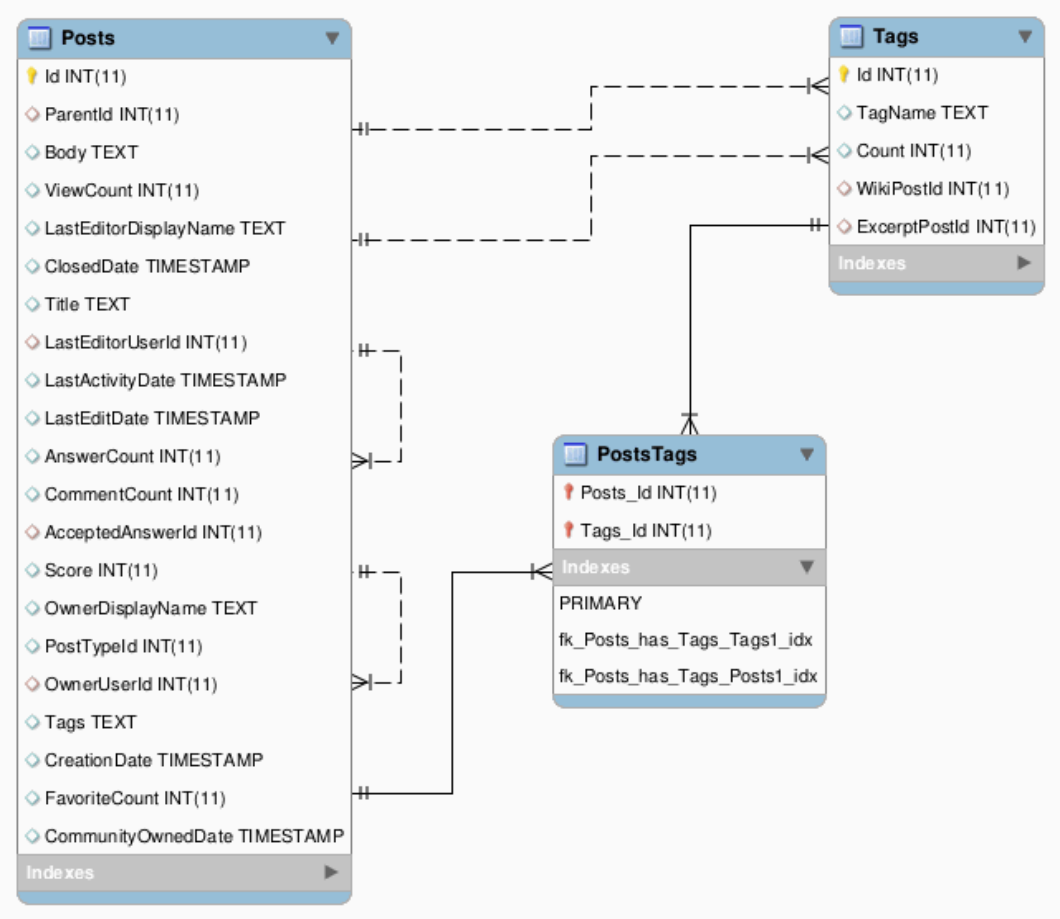
En Stack Overflow esta relación está desnormalizada (más adelante). Sin embargo, podemos construir la relación que existe entre **Posts** y **Tags**: sería una relación **n:m**



Relaciones n:m (ii)

- Conceptualmente, es una relación ***n:m***
- Un Post puede tener muchos Tags
- Y un Tag puede aparecer en muchos Posts
- ¿Cómo se obtendría?

Relaciones n:m — tabla `PostTags`



Relaciones n:m (iii)

- **Ahora bien**:
  - El modelo relacional **no permite añadir esa relación tal cual**
  - Requiere del uso de una **entidad artificial** (*gap* semántico)
- ¿Con qué se corresponde la tabla `PostTags` en el mundo real?
- Ahora la relación es ***identifying*** 🙂
- El caso anterior **1:n** es un caso especial de éste, por lo que también se podría implementar igual

Relaciones dependientes *vs.* no dependientes

- Son más comunes las **no dependientes**
- Generalmente las relaciones **dependientes** se dan en casos de relaciones **1:n** (p. ej. Posts/Comments)
  - La entidad «1» se considera la entidad «**padre**» (**Posts**)
  - Y la otra, la entidad dependiente (**Comments**)
- La entidad dependiente está supeditada a la entidad padre

Relaciones dependientes (ii)

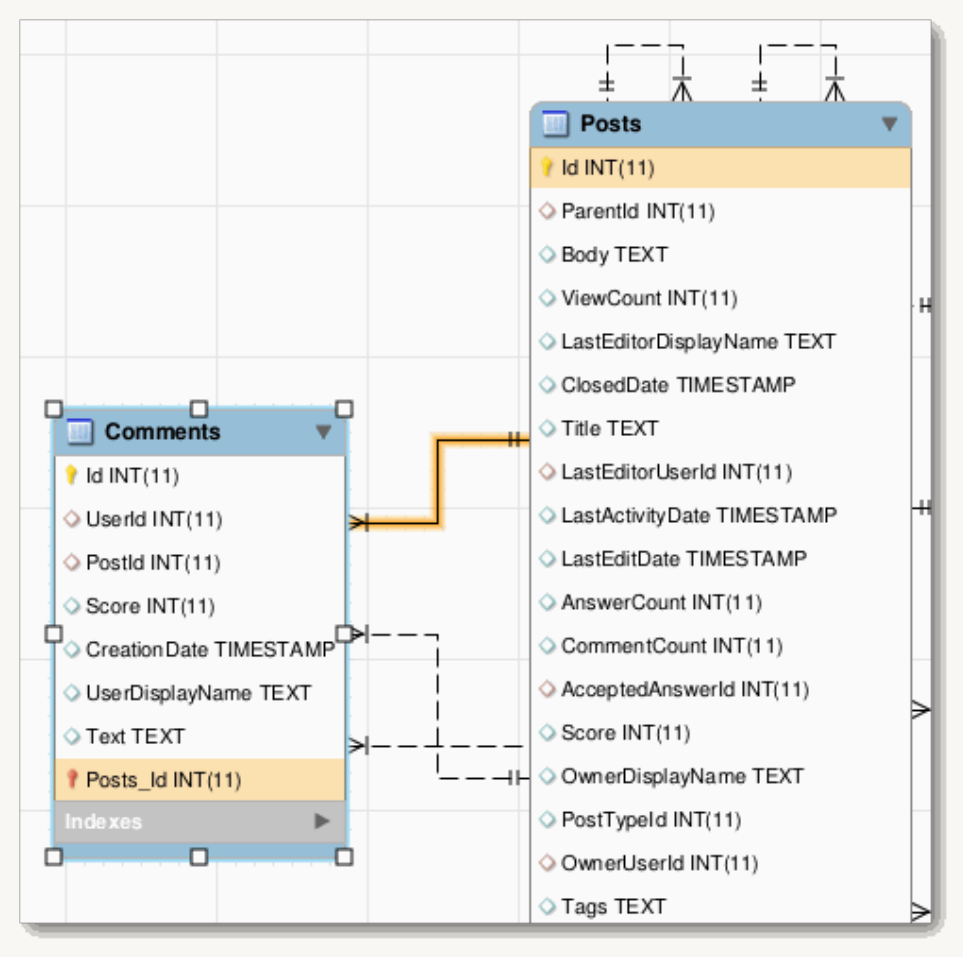
- Su tiempo de vida también está supeditado:
  - Cuando la entidad padre se elimina, la entidad dependiente no tiene sentido por sí misma
  - Generalmente indica una restricción de clave ajena con cascada en *delete* y *update*
- Estrategias de implementación relacional:
  - Una tabla aparte (el caso **1:n** es un caso particular de **n:m**)
  - Una clave ajena en la entidad **n** (dependiente o no) que referencia a la entidad **1**
- En las relaciones **dependientes**, dos alternativas:

i. Clave primaria independiente para la entidad dependiente + clave ajena

ii. Clave primaria que incluye la clave dependiente y la clave ajena

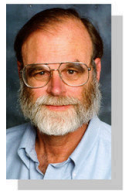
Relaciones dependientes (1:n)

- La clave primaria de **Comments incluye** a la clave ajena de **Posts**
- La clave primaria es (Id, Posts_Id)
- La identificación de un **Comment** sólo tiene sentido **dentro** de un Post
- Al eliminar un Post, el comentario no tiene sentido
- P. ej.: Post 1000, Comentario 1: (clave ⟨1,1000⟩)



Eficiencia

Data Access Hitting a Wall



Current practice based on data download (FTP/GREP)
Will not scale to the datasets of tomorrow

- You can GREP 1 MB in a second
- You can GREP 1 GB in a minute
- You can GREP 1 TB in 2 days
- You can GREP 1 PB in 3 years.

- Oh!, and 1PB ~5,000 disks

- At some point you need
  indices to limit search
  parallel data search and analysis
- This is where databases can help

- You can FTP 1 MB in 1 sec
- You can FTP 1 GB / min (~1$)
- … 2 days and 1K$
- … 3 years and 1M$



[slide src: Jim Gray]

Agregar antes de unir relaciones 1:n

Una publicación puede tener varios comentarios y votos. Unir ambas relaciones directamente multiplica sus filas:

```
1 publicación × 3 comentarios × 4 votos = 12 filas
```

Aplicar `COUNT()` o `SUM()` después contaría varias veces los mismos hechos. La solución es agregar cada relación por separado hasta obtener **una fila por publicación** y unir después esos resultados. Si la granularidad buscada fuese una fila por pregunta incluyendo sus respuestas, cada agregado tendría que reducirse primero a `PreguntaId`

Agregar antes de unir relaciones 1:n (consulta)

```sql
WITH C AS (
  SELECT PostId, COUNT(*) AS NumComentarios
  FROM Comments GROUP BY PostId
), V AS (
  SELECT PostId, COUNT(*) AS NumVotos
  FROM Votes GROUP BY PostId
)
SELECT p.Id, COALESCE(c.NumComentarios, 0), COALESCE(v.NumVotos, 0)
FROM Posts AS p
LEFT JOIN C AS c ON c.PostId = p.Id
LEFT JOIN V AS v ON v.PostId = p.Id;
```

Con nombres distintos (`Id` frente a `PostId`) hace falta `ON` ; si cada CTE expone la clave igual, `USING (PostId)` evita repetirla en cada unión

Índices

- Sea con SQL o con datos *raw*, la búsqueda no se puede realizar secuencialmente
- Los ***índices*** se utilizan para acelerar esta búsqueda
- El concepto de índice está presente en casi todas las bases de datos (también es un eje principal en las NoSQL)
- En SQL, los índices se pueden aplicar a los valores de **una o más columnas**
- El planificador de consultas usa los índices disponibles; es **labor del usuario** definir los adecuados
- Los índices aceleran las búsquedas, pero también ocupan espacio
- Se utiliza `CREATE INDEX`, `ALTER TABLE` o se incluyen en `CREATE TABLE`

Índices: `EXPLAIN`

`EXPLAIN` muestra si una consulta usa un índice:

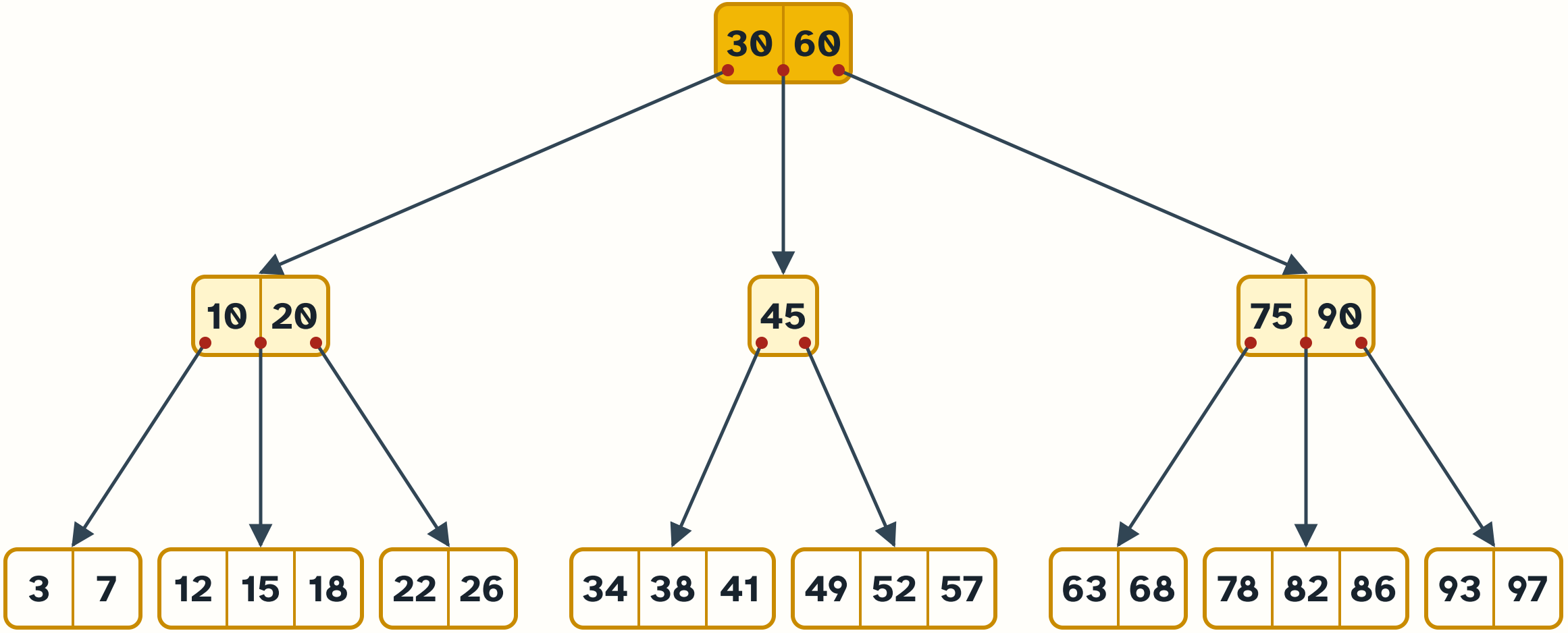
```
mysql> explain select * from Posts where Id=5;
+----+-------------+-------+-------+-------------------+---------+ ...
| id | select_type | table | type  | possible_keys | key     | ...
+----+-------------+-------+-------+-------------------+---------+ ...
|  1 | SIMPLE      | Posts | const | PRIMARY       | PRIMARY | ...
+----+-------------+-------+-------+-------------------+---------+ ...
```

En general, hay tres tipos de índices: basados en árboles balanceados, basados en tablas *hash* e índices *full-text*

Índices de árboles balanceados

- Mantienen la clave ordenada dentro de un árbol balanceado (B-Tree, B+, etc.)
- Aceleran búsquedas con los operadores `=`, `<` y `>`, y rangos (`BETWEEN`)
- También `LIKE` que no empiece con comodín
- Puede ayudar en `ORDER BY`

Índices de árboles balanceados: B-tree



Índices compuestos y prefijo izquierdo

Un índice B-tree compuesto se ordena lexicográficamente por sus columnas:

```
CREATE INDEX idx_autor_fecha
ON Posts (OwnerUserId, CreationDate);
```

Puede localizar eficientemente consultas por:

- `OwnerUserId`
- `OwnerUserId` y `CreationDate`

En general, no permite localizar directamente sólo por `CreationDate`: falta la primera columna del índice. Es la **regla del prefijo izquierdo**

El orden debe elegirse a partir de los patrones de acceso, no del orden en que las columnas aparecen escritas en `WHERE`

Igualdades antes del primer rango

Con un índice `(A, B, C)`, el caso más favorable encadena igualdades y después un rango:

```
WHERE A = 7 AND B = 3 AND C ≥ 100
```

El árbol puede acotar el intervalo mediante las tres columnas. En cambio:

```
WHERE A = 7 AND B ≥ 3 AND C = 100
```

el rango sobre `B` suele ser el último que acota el recorrido. `C` todavía puede evaluarse desde el índice —por ejemplo, mediante *index condition pushdown*—, pero normalmente no reduce el intervalo que hay que recorrer

Por eso suelen colocarse primero las columnas comparadas por igualdad y después la columna de rango, siempre contrastándolo con las consultas reales

Predicados sargables

Un predicado es **sargable** cuando el motor puede convertirlo en un intervalo de búsqueda sobre el valor almacenado en el índice:

```
-- Sargable
WHERE CreationDate ≥ '2016-01-01'
  AND CreationDate <  '2017-01-01'

-- Mismo criterio lógico, pero no sargable sobre CreationDate
WHERE YEAR(CreationDate) = 2016
```

Envolver la columna indexada en una función obliga normalmente a calcular la función por fila. También pueden impedir el acceso eficiente conversiones de tipo implícitas, operaciones aritméticas sobre la columna o patrones `LIKE '%texto'`

La alternativa es reescribir el predicado sobre la columna original o crear, si el patrón lo justifica, una columna generada con su propio índice

Índice candidato no significa índice eficiente

- Un índice es **candidato** si sus columnas y operadores permiten usarlo para la consulta; aparece normalmente en `possible_keys` de `EXPLAIN`
- El optimizador puede no elegirlo si estima que recuperará gran parte de la tabla, si otro índice es más selectivo o si el acceso aleatorio costaría más que un recorrido secuencial
- Que aparezca en `key` confirma la elección, no que el plan sea bueno
- Un índice de cobertura puede ahorrar el acceso a las filas de la tabla, pero ocupa más espacio y encarece las escrituras
- `EXPLAIN ANALYZE` ejecuta la consulta y permite contrastar estimaciones, filas recorridas, bucles y tiempo real

La decisión se justifica con el patrón de consulta, la distribución de los datos y el plan medido, no sólo con la existencia del índice

`EXPLAIN`: `rows` y `filtered`

```
type | key  |  rows | filtered | Extra
ALL  | NULL |  2923 |    11.11 | Using where
```

- `rows`: filas que el motor **estima que tendrá que leer** para resolver ese paso
- `filtered`: porcentaje **estimado** de esas filas que cumplen las condiciones de la tabla (`WHERE`) y siguen adelante; el resto se lee y se descarta
- Filas que pasan al siguiente paso (por ejemplo, un `JOIN`): `rows × filtered / 100`, aquí unas 325 de 2923
- `filtered = 100` significa que casi todo lo leído se aprovecha; un valor bajo, que se lee mucho para quedarse con poco, señal de que falta un índice más selectivo
- Es una estimación a partir de estadísticas, índices e histogramas; sin ellos, el optimizador aplica porcentajes por omisión que no salen de los datos
- `EXPLAIN ANALYZE` no tiene esta columna: mide las filas reales (`actual rows`)

Una tabla para comparar planes

```sql
CREATE TABLE PostsPlanDemo AS
SELECT Id, CreationDate, OwnerUserId, Score, PostTypeId, AnswerCount
FROM Posts
WHERE CreationDate IS NOT NULL
ORDER BY Id
LIMIT 50000;

ALTER TABLE PostsPlanDemo
  ADD PRIMARY KEY (Id),
  ADD INDEX idx_owner_creation (OwnerUserId, CreationDate),
  ADD INDEX idx_creation_owner (CreationDate, OwnerUserId),
  ADD INDEX idx_covering (PostTypeId, CreationDate, Score);
```

Una copia de 50 000 publicaciones con tres índices compuestos. Las diapositivas siguientes lanzan consultas sobre ella y comparan los planes. Las salidas son **aproximadas**: los tiempos dependen de la máquina y están redondeados

Leer `EXPLAIN ANALYZE`

```
EXPLAIN ANALYZE SELECT Id FROM PostsPlanDemo FORCE INDEX (idx_owner_creation)
WHERE OwnerUserId = 95 AND CreationDate ≥ '2016-01-01';
```

```
→ Filter: (OwnerUserId = 95 and CreationDate ≥ '2016-01-01')
  (cost=1.2 rows=1) (actual time=0.01..1.1 rows=1636 loops=1)
  → Covering index range scan on PostsPlanDemo using idx_owner_creation
    over (OwnerUserId = 95 AND '2016-01-01' ≤ CreationDate)
    (cost=1.2 rows=1) (actual time=0.01..0.6 rows=1636 loops=1)
```

- Se lee de dentro afuera: cada `→` es un paso que alimenta al de arriba
- `rows=1` es la **estimación**; `actual ... rows=1636`, las filas **reales**: aquí difieren mucho, así que la estimación es pobre
- `loops`: veces que se ejecutó el paso; `actual time`: ms hasta la primera y la última fila
- `FORCE INDEX` impone un índice: sirve para **experimentar**

El orden de las columnas, en el plan

La misma consulta forzada sobre cada índice:

```
idx_owner_creation (OwnerUserId, CreationDate)
→ Covering index range scan on PostsPlanDemo
   over (OwnerUserId = 95 AND '2016-01-01' ≤ CreationDate)
   (actual time=0.01..0.6 rows=1636 loops=1)

idx_creation_owner (CreationDate, OwnerUserId)
→ Covering index range scan on PostsPlanDemo
   over ('2016-01-01' ≤ CreationDate AND 95 ≤ OwnerUserId)
   (actual time=0.01..20 rows=48243 loops=1)
```

- Mismo resultado (1636 filas), distinto trabajo: 1636 entradas del índice frente a 48 243
- Con la igualdad primero, el árbol acota el intervalo; con el rango primero recorre todas las fechas desde 2016 y descarta después a los demás propietarios
- Para «todas las publicaciones desde una fecha» sería mejor el orden contrario

Sargable frente a no sargable, en el plan

```
WHERE CreationDate ≥ '2016-06-01' AND CreationDate < '2016-07-01'
```

```
→ Covering index range scan on PostsPlanDemo using idx_creation_owner
  over ('2016-06-01' ≤ CreationDate < '2016-07-01')
  (actual time=0.1..0.7 rows=2551 loops=1)
```

```
WHERE YEAR(CreationDate) = 2016 AND MONTH(CreationDate) = 6
```

```
→ Covering index scan on PostsPlanDemo using idx_covering
  (actual time=0.5..6 rows=50000 loops=1)
```

- Mismo resultado (2551 filas): con el rango se leen 2551 entradas del índice; con las funciones, las 50 000
- Un `index scan` sin `over (...)` recorre el índice entero: no hay intervalo que buscar

Índice de cobertura, en el plan

```
SELECT PostTypeId, CreationDate, Score FROM PostsPlanDemo
WHERE PostTypeId = 1 AND CreationDate ≥ '2016-01-01'
ORDER BY CreationDate LIMIT 25;
```

```
→ Covering index range scan on PostsPlanDemo using idx_covering
```

Pidiendo además `AnswerCount`, que no está en `idx_covering`:

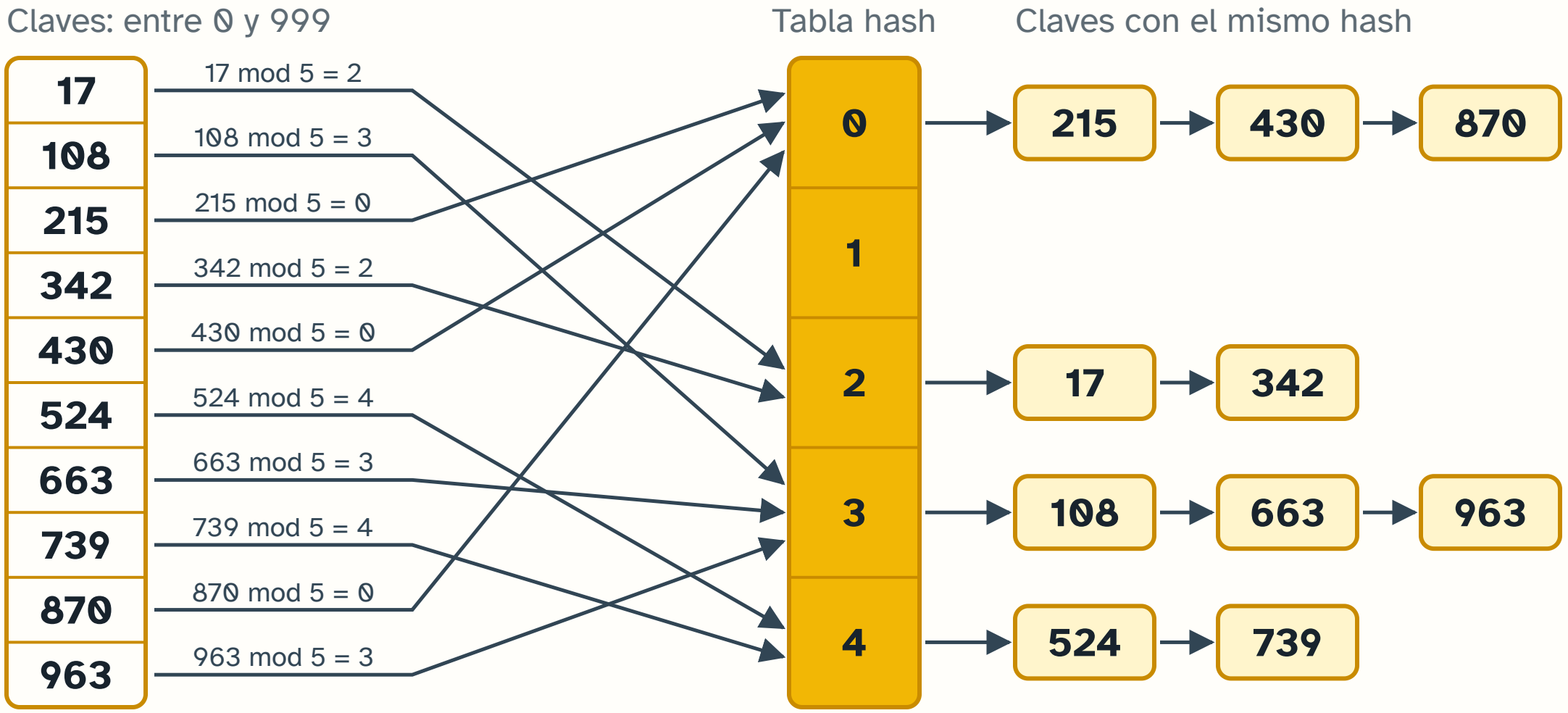
```
→ Index range scan on PostsPlanDemo using idx_covering
    ... , with index condition: (PostTypeId = 1 and CreationDate ≥ ...)
```

- `Covering` significa que el índice contiene todas las columnas que pide la consulta: no se lee la tabla
- Una columna más obliga a leer la fila de la tabla por cada entrada del índice

Índices de tablas *hash*

- Sólo para consultas de igualdad (`=` o `<=>`, la igualdad que trata `NULL` como un valor)
- Son muy rápidos
- No se pueden usar para `ORDER BY` ni para rangos
- Sólo se pueden usar con valores completos de clave (no permiten prefijos, por ejemplo)
- En MySQL sólo existen en los motores `MEMORY` y `NDB`: InnoDB usa siempre B-tree (su *adaptive hash index* es interno y automático)

Índices de tablas *hash*: ejemplo



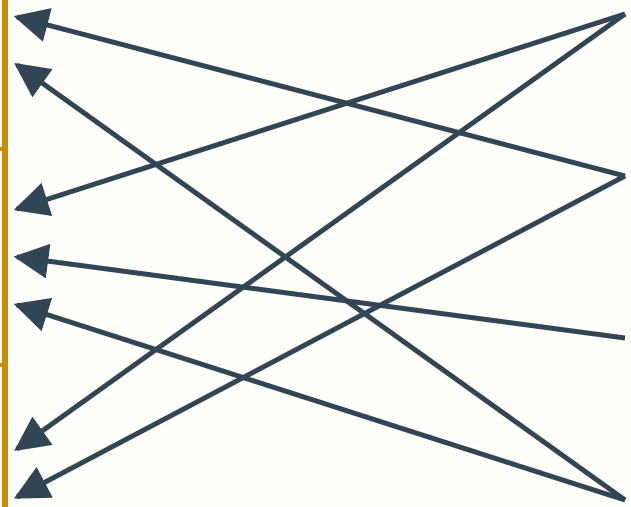
Índices *full-text*

- Usados en búsquedas inversas sobre campos textuales
- Dada una palabra o conjunto de palabras, encuentran la fila o filas en las que el campo indexado contiene esas palabras
- Tienen limitaciones con `LIKE` y comodines (`%`)
- Por ello los SGBD proveen operadores especiales (p. ej. en MySQL `MATCH( ... ) AGAINST ...` , se estudia en prácticas)

Índices *full-text*: índice invertido

Tabla Posts

Id	Title
1	**Índices** en **MySQL**
2	**Consultas lentas** en **MySQL**
3	**Índices** y **consultas**



Índice full-text

Palabra	Filas
consultas	2, 3
índices	1, 3
lentas	2
mysql	1, 2

No se indexan las palabras de menos de 3 letras: «en», «y»

Desnormalización

- El modelo relacional basa el modelado de datos en la **normalización**, en sus distintos niveles
- Esto permite aplicar el principio de «sólo almacenar un dato en un sitio», lo que lleva a la eficiencia de datos
- Pero a veces necesitamos también **eficiencia de acceso**, y a veces está reñida con la normalización
- El principal escollo para la velocidad es la unión (`JOIN`) de datos de tablas

¿Por qué desnormalizar?

- **Mantener la historia**: los datos de los clientes, por ejemplo, pueden cambiar; necesitamos almacenar sus valores, p. ej., al emitir una factura en el pasado. Para eso hay que copiar los datos del cliente en ese momento
- **Mejorar la velocidad de consulta**: a veces, para consultas recurrentes, tenemos que usar uniones de varias tablas. Se puede evitar la unión, por ejemplo, añadiendo claves ajenas a las tablas finales, o bien —como en `Posts.Tags` — replicando la información
- **Precalcular valores necesitados de antemano**: si hay datos que se necesitarán, como medias o totales, podemos calcularlos y almacenarlos de antemano

Desventajas de desnormalizar

- **Espacio en disco**: al igual que con los índices, la copia y duplicación de datos ocupa más espacio
- **Anomalías de datos**: un cambio en un dato puede requerir un cambio en más de una parte de la base de datos para mantener la consistencia. Los SGBDR no están preparados para mantener este tipo de consistencia por sí solos; hay que usar *triggers*, transacciones, etc.
- **Ralentización de las actualizaciones**: dependiendo del *ratio* entre actualizaciones y consultas, puede ser un punto importante
- **Más código, más dependencias, más propenso a errores**: al necesitar más coordinación, hay más dependencias ocultas, más errores y mayor coste de mantenimiento

Desnormalización: ejemplos en Stack Overflow

- En la tabla `Posts` tenemos, por ejemplo, el campo `Tags`, que contiene la lista de los tags. Evita tener que hacer un `JOIN` para obtener sus nombres
- También está el campo `Score`, que aglutina todos los votos positivos recibidos (también está en la tabla `Votes`)

Particiones

- El mecanismo de partición permite partir tablas de forma **lógica**
- Es decir, **transparente al usuario**
- Las tablas partidas tienen ciertas restricciones: por ejemplo, no permiten claves únicas arbitrarias y, en InnoDB, no admiten claves ajenas
- La tabla sigue viviendo en **un único servidor**: repartir datos entre máquinas es *sharding*, que veremos con Cassandra y MongoDB

Particionado y claves únicas en MySQL

Todas las columnas usadas por la expresión de particionado deben pertenecer a **cada clave única** de la tabla, incluida la clave primaria

```sql
CREATE TABLE PostsPorAño (
  Id BIGINT NOT NULL,
  CreationDate DATETIME NOT NULL,
  Score INT,
  PRIMARY KEY (Id, CreationDate)
)
PARTITION BY RANGE (YEAR(CreationDate)) (
  PARTITION p2016 VALUES LESS THAN (2017),
  PARTITION p2017 VALUES LESS THAN (2018),
  PARTITION p_futuro VALUES LESS THAN MAXVALUE
);
```

`PRIMARY KEY (Id)` no sería válida: dos particiones distintas no pueden comprobar por separado una unicidad global sobre `Id`. Esta restricción debe tenerse en cuenta antes de copiar una tabla existente y conservar sus claves

Ventajas del particionado

- Aceleran las búsquedas que filtran por la columna de particionado, porque el optimizador puede descartar particiones (*partition pruning*)
- Aceleran las búsquedas de rangos sobre esa columna
- Permiten almacenar la información histórica de forma eficiente (queda en particiones que se usan rara vez, mientras que los datos actuales usan sólo las últimas particiones)
- Facilitan el mantenimiento: borrar un año entero con `DROP PARTITION` es inmediato frente a un `DELETE` fila a fila

Inconvenientes del particionado

- Cada clave única debe incluir las columnas de particionado
- Las escrituras no se aceleran: cada inserción debe calcular su partición, y los índices son locales a cada partición
- Una consulta que no filtra por la columna de particionado recorre **todas** las particiones y puede resultar más costosa que sin particionar
- Más complejidad de diseño y de mantenimiento (crear particiones nuevas a tiempo, reorganizar)

¿Cuándo usar particionado?

- Si casi siempre se hacen consultas sobre la columna de particionado
- Si se tienen datos históricos que se quiere que no «estorben»
- Si los índices son más grandes que la memoria RAM

En MySQL se especifican en `CREATE TABLE` o `ALTER TABLE`:

```
CREATE TABLE <table_name> (<columns>)
PARTITION BY <type> (<partition_expression>);
```

`<type>` puede ser `RANGE`, `LIST`, `HASH` o `KEY`; `RANGE COLUMNS` y `LIST COLUMNS` particionan directamente sobre una o varias columnas, sin expresión

Particionado `RANGE`

```
CREATE TABLE  ...
PARTITION BY RANGE (att) (
PARTITION p0 VALUES LESS THAN (10000),
PARTITION p1 VALUES LESS THAN (20000),
PARTITION p2 VALUES LESS THAN (30000),
PARTITION p3 VALUES LESS THAN (40000),
PARTITION p4 VALUES LESS THAN MAXVALUE)
```

Particionado `LIST`

```
CREATE TABLE  ...
  PARTITION BY LIST (store_id) (
PARTITION pNorth VALUES IN (2,8,12),
PARTITION pEast VALUES IN (1,4,7),
PARTITION pWest VALUES IN (3,5,6,10),
PARTITION pSouth VALUES IN (9,11)
)
```

Particionado `HASH` y `KEY`

```
CREATE TABLE  ...
PARTITION BY HASH (att)
PARTITIONS 4;
```

```
CREATE TABLE  ...
PARTITION BY KEY (att)
PARTITIONS 4;
```

Consultar particiones

```
-- En MySQL 8, la salida normal ya incluye la columna `partitions`.
EXPLAIN SELECT  ...
```

```
SELECT * FROM Tabla PARTITION (p0, p1,  ...) WHERE  ...
```

Partition pruning: comprobar, no suponer

El optimizador descarta particiones cuando puede deducir de `WHERE` cuáles podrían contener filas. Para particiones anuales, un rango directo suele expresar bien esa información:

```
EXPLAIN SELECT COUNT(*) FROM PostsPorAño
WHERE CreationDate ≥ '2016-01-01'
  AND CreationDate <  '2017-01-01';
```

Una transformación arbitraria puede impedir esa deducción:

```
EXPLAIN SELECT COUNT(*) FROM PostsPorAño
WHERE DATE_FORMAT(CreationDate, '%Y') = '2016';
```

Ni siquiera una función idéntica a la expresión de particionado, como `YEAR(CreationDate) = 2016`, garantiza el recorte. Por eso no basta con razonar por intuición: la columna `partitions` de `EXPLAIN` indica las particiones que realmente se leerán

Partition pruning: la columna `partitions`

`EXPLAIN` sobre `PostsPorAño` (`p2016`, `p2017` y `p_futuro`), en MySQL 8.4:

Predicado	`partitions`
`CreationDate ≥ '2016-01-01' AND CreationDate < '2017-01-01'`	`p2016`
`DATE_FORMAT(CreationDate, '%Y') = '2016'`	`p2016,p2017,p_futuro`
`YEAR(CreationDate) = 2016`	`p2016,p2017,p_futuro`

- Sólo el rango sobre la propia columna se traduce a una partición
- El tercero es el engañoso: coincide con la expresión de particionado y aun así se leen todas
- Comprobarlo en `EXPLAIN` es la única forma de saberlo

Subparticiones

Se pueden combinar dos niveles: `RANGE` o `LIST` en el primero y `HASH` o `KEY` en el segundo

```
CREATE TABLE PostsPart_Mixed ( ... )
PARTITION BY RANGE (YEAR(CreationDate))
SUBPARTITION BY HASH (Id)
SUBPARTITIONS 2 (
  PARTITION p2014 VALUES LESS THAN (2015),
  PARTITION p2015 VALUES LESS THAN (2016),
  PARTITION p_futuro VALUES LESS THAN MAXVALUE
);
```

- El año permite descartar particiones; el *hash* reparte cada año en trozos de tamaño parecido
- Cada subpartición es una partición física más: 3 × 2 = 6 en el ejemplo

Particionar por mes: `RANGE COLUMNS`

Con `PARTITION BY RANGE (YEAR(CreationDate)*100 + MONTH(CreationDate))`, un rango de fechas lee **todas** las particiones: MySQL sólo traduce rangos a particiones con la columna directa o con funciones que sabe crecientes (`YEAR()`, `TO_DAYS()`, `TO_SECONDS()`), y esta expresión no lo es

`RANGE COLUMNS` usa la propia columna, con fechas como límites:

```
PARTITION BY RANGE COLUMNS (CreationDate) (
   ...
  PARTITION p201403 VALUES LESS THAN ('2014-04-01'),
  PARTITION p201404 VALUES LESS THAN ('2014-05-01'),
   ...
  PARTITION p_futuro VALUES LESS THAN (MAXVALUE)
);
```

Ahora `WHERE CreationDate ≥ '2014-03-01' AND CreationDate < '2014-05-01'` sólo lee `p201403` y `p201404`

Mantenimiento de particiones

Partir la última partición para abrir un año nuevo:

```sql
ALTER TABLE PostsPorAño
REORGANIZE PARTITION p_futuro INTO (
  PARTITION p2018 VALUES LESS THAN (2019),
  PARTITION p_futuro VALUES LESS THAN MAXVALUE
);
```

Purgar un año entero y consultar los metadatos:

```sql
ALTER TABLE PostsPorAño DROP PARTITION p2016;

SELECT PARTITION_NAME, TABLE_ROWS, PARTITION_DESCRIPTION
FROM INFORMATION_SCHEMA.PARTITIONS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'PostsPorAño';
```

`TABLE_ROWS` es una estimación, no un recuento exacto

Cuestiones avanzadas

Transacciones

- Una **transacción** agrupa modificaciones que se confirman o deshacen juntas
- InnoDB es **siempre** transaccional: con `autocommit` activo (por defecto), cada sentencia es una transacción que se confirma sola
- Para agrupar varias sentencias:

```
START TRANSACTION;
UPDATE Cuentas SET Saldo = Saldo - 100 WHERE Id = 1;
UPDATE Cuentas SET Saldo = Saldo + 100 WHERE Id = 2;
COMMIT;       -- o ROLLBACK; para deshacer ambos
```

- Las sentencias DDL (`CREATE` , `ALTER` , `DROP`) provocan un `COMMIT` implícito en MySQL: no se pueden deshacer con `ROLLBACK`

Funciones ventana: `OVER (...)`

Calcula un valor **por fila** a partir de su grupo, sin colapsarlo como `GROUP BY`:

```sql
SELECT MonthStart, OwnerUserId, NumQuestions,
       ROW_NUMBER() OVER (
         PARTITION BY MonthStart
         ORDER BY NumQuestions DESC, OwnerUserId
       ) AS rn
FROM MonthlyQuestions;
```

- `PARTITION BY` define los grupos (un mes); `ORDER BY`, el orden en cada uno
- `ROW_NUMBER()` numera 1, 2, 3…; `RANK()` repite el puesto en los empates y deja huecos (1, 1, 3); `DENSE_RANK()` no deja huecos (1, 1, 2)
- Se evalúan tras `WHERE` y `GROUP BY`: para filtrar por `rn` hay que envolver la consulta en una CTE

CTE y ventana juntas: los cinco autores de cada mes

```sql
WITH MonthlyQuestions AS (
  SELECT DATE_FORMAT(CreationDate, '%Y-%m-01') AS MonthStart,
         OwnerUserId, COUNT(*) AS NumQuestions
  FROM Posts
  WHERE PostTypeId = 1 AND OwnerUserId IS NOT NULL
  GROUP BY MonthStart, OwnerUserId
), Ranked AS (
  SELECT MonthStart, OwnerUserId, NumQuestions,
         ROW_NUMBER() OVER (PARTITION BY MonthStart
           ORDER BY NumQuestions DESC, OwnerUserId) AS rn
  FROM MonthlyQuestions
)
SELECT * FROM Ranked WHERE rn ≤ 5 ORDER BY MonthStart, rn;
```

CTE y ventana juntas: el resultado

```
MonthStart  OwnerUserId  NumQuestions  rn
2015-10-01           23             2   1
2015-10-01           24             2   2
2015-10-01           37             2   3
2015-10-01           20             1   4
2015-10-01           21             1   5
2015-11-01           21             5   1
...
```

- La primera CTE fija la granularidad: una fila por mes y autor
- La segunda numera dentro de cada mes, con el desempate por `OwnerUserId`
- El filtro `rn ≤ 5` va fuera porque `rn` no existe aún en el `SELECT` que la calcula

Estadísticas e histogramas

- El optimizador elige el plan a partir de **estadísticas**: número de filas, cardinalidad de los índices, distribución de valores
- `ANALYZE TABLE Posts` recalcula las estadísticas de los índices
- Con una distribución **sesgada** (pocas etiquetas muy frecuentes, muchos usuarios con una sola pregunta) suponer valores uniformes lleva a malas estimaciones
- Un **histograma** describe la distribución de una columna, aunque no esté indexada:

```
ANALYZE TABLE Posts
UPDATE HISTOGRAM ON PostTypeId, AnswerCount WITH 32 BUCKETS;
```

- No acelera nada por sí mismo: sólo mejora las estimaciones de `rows` y `filtered` y, con ello, la elección del plan